

NAME: J SRAVANI

REG NO: 192372141

COURSE CODE: CSA1403

COURSE NAME: COMPLIER DESIGN

1. The lexical analyzer should ignore redundant spaces, tabs, and new lines. It should also ignore comments. Although the syntax specification states that identifiers can be arbitrarily long, you may restrict the length to some reasonable value. Develop a lexical Analyzer to identify identifiers, constants, and operators using the C program.

CODE

```
#include <stdio.h>
#include <ctype.h>
int main() {
    char s[50];
    int i;
    printf("Enter: ");
    scanf("%s", s);
    for(i=0; s[i]!='\0'; i++) {
        if(isalpha(s[i]))
            printf("%c - Identifier\n",s[i]);
        else if(isdigit(s[i]))
            printf("%c - Constant\n",s[i]);
        else if(s[i]=='+'||s[i]=='-'||s[i]=='*'||s[i]=='/')
            printf("%c - Operator\n",s[i]);
        }
    return 0;
}
```

OUTPUT

Enter: a+5*b

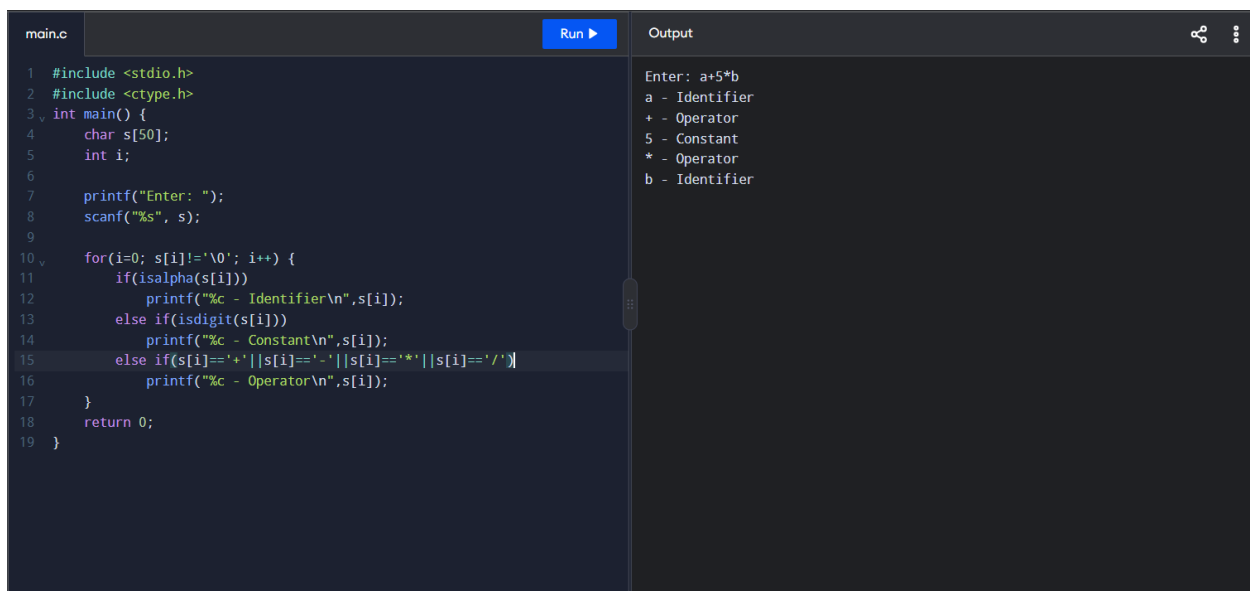
a - Identifier

+ - Operator

5 - Constant

* - Operator

b - Identifier



```
main.c Run ▶ Output
1 #include <stdio.h>
2 #include <ctype.h>
3 int main() {
4     char s[50];
5     int i;
6
7     printf("Enter: ");
8     scanf("%s", s);
9
10    for(i=0; s[i]!='\0'; i++) {
11        if(isalpha(s[i]))
12            printf("%c - Identifier\n", s[i]);
13        else if(isdigit(s[i]))
14            printf("%c - Constant\n", s[i]);
15        else if(s[i]=='+'||s[i]=='-'||s[i]=='*'||s[i]=='/')
16            printf("%c - Operator\n", s[i]);
17    }
18    return 0;
19 }
```

Enter: a+5*b
a - Identifier
+ - Operator
5 - Constant
* - Operator
b - Identifier

2. Extend the lexical Analyzer to Check comments, defined as follows in C:

a) A comment begins with // and includes all characters until the end of that line.

b) A comment begins with /* and includes all characters through the next occurrence of the character sequence */Develop a lexical Analyzer to identify whether a given line is a comment or not.

CODE

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[200];
```

```

printf("Enter a line: ");
fgets(str, sizeof(str), stdin);
if (str[0] == '/' && str[1] == '/')
    printf("It is a single-line comment.\n");
else if (str[0] == '/' && str[1] == '*')
    printf("It is a multi-line comment.\n");
else
    printf("It is not a comment.\n");
return 0;
}

```

OUTPUT

Enter a line: // hello world

It is a single-line comment.

```

main.c Run ▶ Output
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char str[200];
6
7     printf("Enter a line: ");
8     fgets(str, sizeof(str), stdin);
9
10    if (str[0] == '/' && str[1] == '/')
11        printf("It is a single-line comment.\n");
12    else if (str[0] == '/' && str[1] == '*')
13        printf("It is a multi-line comment.\n");
14    else
15        printf("It is not a comment.\n");
16
17    return 0;
18 }

```

Enter a line: // hello world
It is a single-line comment.

3. Design a lexical Analyzer for a given language should ignore the redundant spaces, tabs, and new lines and ignore comments.

CODE

```
#include <stdio.h>

int main() {
    char str[200];
    int i = 0;
    printf("Enter a line: ");
    fgets(str, sizeof(str), stdin);
    while (str[i] != '\0') {
        if (str[i] == '/' && str[i + 1] == '/') {
            break;
        }
        if (str[i] != ' ' && str[i] != '\t' && str[i] != '\n')
            printf("%c", str[i]);
        i++;
    }
    printf("\n");
    return 0;
}
```

OUTPUT

Enter a line: int a = 10

inta=10

```
main.c Run ▶ Output
1 #include <stdio.h>
2
3 int main() {
4     char str[200];
5     int i = 0;
6
7     printf("Enter a line: ");
8     fgets(str, sizeof(str), stdin);
9
10    while (str[i] != '\0') {
11        if (str[i] == '/' && str[i + 1] == '/') {
12            break;
13        }
14
15        if (str[i] != ' ' && str[i] != '\t' && str[i] != '\n')
16            printf("%c", str[i]);
17
18        i++;
19    }
20
21    printf("\n");
22    return 0;
23 }
```

Enter a line: int a = 10
inta=10

4. Design a lexical Analyzer to validate operators to recognize the operators +,-,*,/ using regular Arithmetic operators.

CODE

```
#include <stdio.h>

int main() {
    char ch;

    printf("Enter an operator: ");
    scanf("%c", &ch);

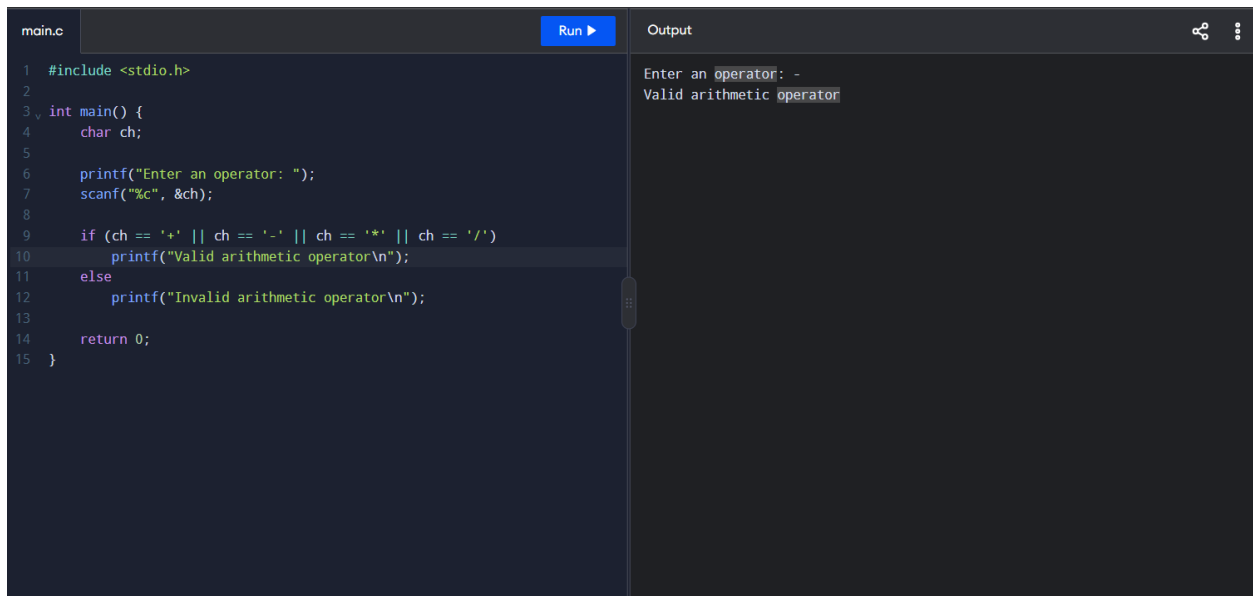
    if (ch == '+' || ch == '-' || ch == '*' || ch == '/')
        printf("Valid arithmetic operator\n");
    else
        printf("Invalid arithmetic operator\n");

    return 0;
}
```

OUTPUT

Enter an operator: -

Valid arithmetic operator



The screenshot shows a C program in a code editor with a dark theme. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     char ch;
5
6     printf("Enter an operator: ");
7     scanf("%c", &ch);
8
9     if (ch == '+' || ch == '-' || ch == '*' || ch == '/')
10        printf("Valid arithmetic operator\n");
11    else
12        printf("Invalid arithmetic operator\n");
13
14    return 0;
15 }
```

The output window on the right shows the program's execution: "Enter an operator: -" followed by "Valid arithmetic operator". A blue "Run" button is visible above the code editor.

5. Design a lexical Analyzer to find the number of whitespaces and newline characters.

CODE

```
#include <stdio.h>

int main() {
    char s[100];
    int i, w = 0, n = 0;
    printf("Enter text:\n");
    fgets(s, sizeof(s), stdin);
    for(i = 0; s[i] != '\0'; i++) {
        if(s[i] == ' ' || s[i] == '\t')
            w++;
        if(s[i] == '\n')
            n++;
    }
}
```

```
    printf("Whitespaces = %d\n", w);

    printf("Newlines = %d\n", n);

    return 0;

}
```

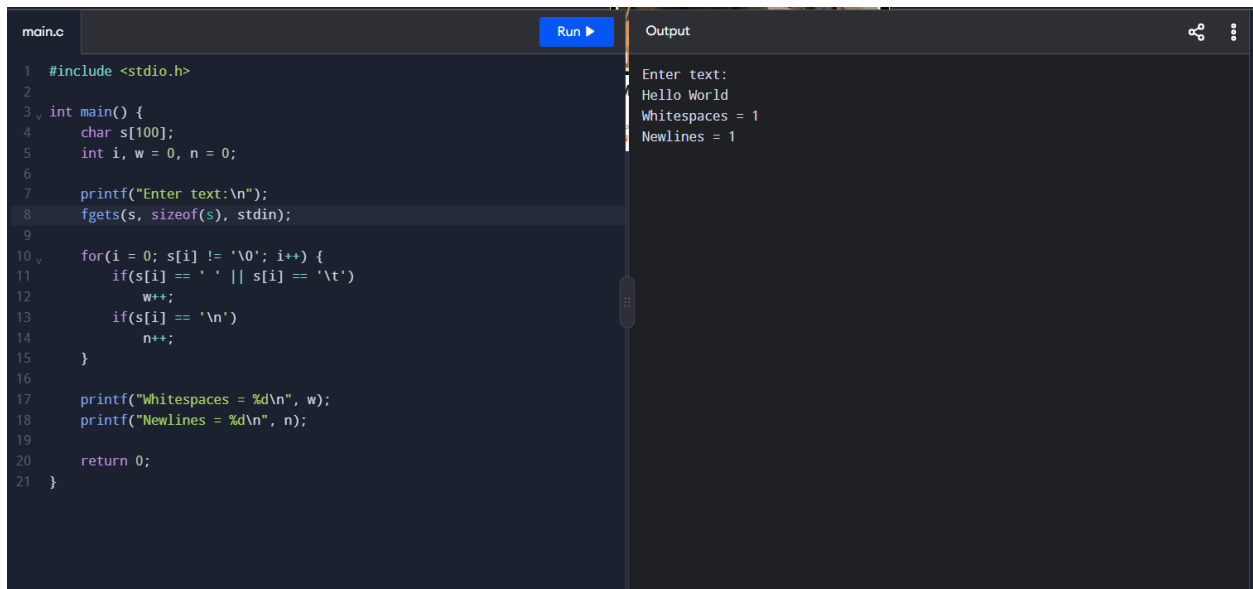
OUTPUT

Enter text:

Hello World

Whitespaces = 1

Newlines = 1

A screenshot of a code editor window with a dark theme. The left pane shows the source code for a file named 'main.c'. The code includes <stdio.h>, defines a main function, declares a char array 's' of size 100, and initializes variables 'i', 'w', and 'n' to 0. It prompts the user to 'Enter text:' using printf, reads the input with fgets, and then iterates through the string 's' to count spaces (incrementing 'w') and newlines (incrementing 'n'). Finally, it prints the counts using printf and returns 0. The right pane, titled 'Output', shows the program's execution: 'Enter text:', 'Hello World', 'Whitespaces = 1', and 'Newlines = 1'. A 'Run' button is visible above the code editor.

```
main.c Run ▶ Output
1  #include <stdio.h>
2
3  int main() {
4      char s[100];
5      int i, w = 0, n = 0;
6
7      printf("Enter text:\n");
8      fgets(s, sizeof(s), stdin);
9
10     for(i = 0; s[i] != '\0'; i++) {
11         if(s[i] == ' ' || s[i] == '\t')
12             w++;
13         if(s[i] == '\n')
14             n++;
15     }
16
17     printf("Whitespaces = %d\n", w);
18     printf("Newlines = %d\n", n);
19
20     return 0;
21 }
```

Enter text:
Hello World
Whitespaces = 1
Newlines = 1

